



UNIVERSIDAD POLITÉCNICA DE PACHUCA

INGENIERÍA EN SOFTWARE

ARQUITECTURA ORIENTADA A SERVICIOS

Grupo: 09_02

MVC con SOAP a bd

Nombre del Alumno: Alvaro Lechuga Vecino

CONSTRUCCIÓN DEL PROYECTO

Parte 1 — Backend (Proyecto WSBD)

Creamos un servicio web SOAP que funciona como intermediario entre la aplicación y la base de datos.

Se compone de dos paquetes:

Paquete Funcionalidad — maneja la base de datos directamente con 4 clases:

- Conexion.java — establece la conexión a MySQL con usuario, contraseña y URL

```
import java.sql.Connection;
import java.sql.DriverManager;

public class Conexion {
    private String driver = "com.mysql.cj.jdbc.Driver"; // driver moderno
    private String url = null;
    private String password = null;
    private String user = null;

    public Conexion(String driver, String url, String user, String password) {
        this.driver = driver;
        this.url = url;
        this.user = user;
        this.password = password;
    }

    public Connection ObtenerConexion() {
        Connection con = null;
        try {
            Class.forName(driver);
            con = DriverManager.getConnection(url, user, password);
        } catch (Exception e) {
            System.out.println("Error en conexión: " + e.getMessage());
        }
        return con;
    }

    // Getters y Setters
    public String getDriver() { return driver; }
    public String getURL() { return url; }
    public String getPassword() { return password; }
    public String getUser() { return user; }
    public void setDriver(String driver) { this.driver = driver; }
    public void setURL(String url) { this.url = url; }
    public void setPassword(String password) { this.password = password; }
    public void setUser(String user) { this.user = user; }
}
```

- MDB.java — ejecuta las operaciones SQL (insertar, eliminar, modificar, consultar)

```
5 package Funcionalidad;
6
7 /**
8  *
9  * @author SirKy
10 */
11
12 import java.sql.*;
13 import java.util.ArrayList;
14 import java.util.List;
15
16 @
17 public class MDB {
18     protected Conexion conexion;
19     protected ResultSet dataSet;
20     protected Statement executer;
21     protected String table;
22
23     public MDB() {
24         // *** MODIFICA ESTOS 3 DATOS CON LOS TUYOS ***
25         conexion = new Conexion(
26             "com.mysql.cj.jdbc.Driver",
27             "jdbc:mysql://localhost:3306/bddatos?useSSL=false&serverTimezone=UTC",
28             "root", // tu usuario MySQL
29             "" // tu contraseña MySQL
30         );
31     }
32
33     public String registrarAlta(String tabla, String valores) {
34         String leyenda = "";
35         try {
36             Connection con = conexion.ObtenerConexion();
37             executer = con.createStatement();
38             String sql = "INSERT INTO " + tabla + " VALUES (" + valores + ")";
39             executer.executeUpdate(sql);
40             leyenda = "Datos almacenados exitosamente.";
41             con.close();
42         } catch (Exception e) {
43             leyenda = e.toString();
44         }
45         return leyenda;
46     }
47 }
```

```

47 public String borrarRegistro(String tabla, String condicion) {
48     String leyenda = "";
49     try {
50         Connection con = conexion.ObtenerConexion();
51         executer = con.createStatement();
52         String sql = "DELETE FROM " + tabla + " WHERE " + condicion;
53         executer.executeUpdate(sql);
54         leyenda = "Registro eliminado exitosamente.";
55         con.close();
56     } catch (Exception e) {
57         leyenda = e.toString();
58     }
59     return leyenda;
60 }
61
62 public String modificarRegistro(String tabla, String datos, String condicion) {
63     String leyenda = "";
64     try {
65         Connection con = conexion.ObtenerConexion();
66         executer = con.createStatement();
67         String sql = "UPDATE " + tabla + " SET " + datos + " WHERE " + condicion;
68         executer.executeUpdate(sql);
69         leyenda = "Registro modificado exitosamente.";
70         con.close();
71     } catch (Exception e) {
72         leyenda = e.toString();
73     }
74     return leyenda;
75 }
76
77 public List<List<String>> consultaDatos(String tabla, String condicion) {
78     List<List<String>> resultado = new ArrayList<>();
79     try {
80         Connection con = conexion.ObtenerConexion();
81         executer = con.createStatement();
82         String sql = "SELECT * FROM " + tabla + " WHERE " + condicion;
83         dataSet = executer.executeQuery(sql);
84         ResultSetMetaData meta = dataSet.getMetaData();
85         int cols = meta.getColumnCount();
86         while (dataSet.next()) {

```

```

        public List<List<String>> consultaDatos(String tabla, String condicion) {
            List<List<String>> resultado = new ArrayList<>();
            try {
                Connection con = conexion.ObtenerConexion();
                executer = con.createStatement();
                String sql = "SELECT * FROM " + tabla + " WHERE " + condicion;
                dataSet = executer.executeQuery(sql);
                ResultSetMetaData meta = dataSet.getMetaData();
                int cols = meta.getColumnCount();
                while (dataSet.next()) {
                    List<String> fila = new ArrayList<>();
                    for (int i = 1; i <= cols; i++) {
                        fila.add(dataSet.getString(i));
                    }
                    resultado.add(fila);
                }
                con.close();
            } catch (Exception e) {
                System.out.println(e.toString());
            }
            return resultado;
        }
    }
}

```

- MDBExtendido.java — extiende MDB para agregar métodos adicionales si se necesitan

```
package Funcionalidad;

/**
 *
 * @author SirKy
 */
public class MDBExtendido extends MDB {

    public MDBExtendido() {
        super();
    }

    public String modificarRegistro(String tabla, String datos,
                                   String condicion, int id) {
        return super.modificarRegistro(tabla, datos, condicion + id);
    }
}
```

- Tester.java — clase de prueba para verificar que la conexión funciona

```
package Funcionalidad;

/**
 *
 * @author SirKy
 */
public class Tester {

    public static void main(String[] args) {
        MDB administradorBD = new MDB();
        // Prueba de alta — ajusta los valores a tu tabla
        String leyenda = administradorBD.registrarAlta(
            "tbusuarios",
            "null, 'Juan', 'Pérez', 'González', 1234"
        );
        System.out.println(leyenda);
    }
}
```

Paquete WSSoap — expone los métodos como servicio web con 4 operaciones:

- altas() — inserta un registro nuevo
- baja() — elimina un registro por condición
- modifica() — actualiza un registro existente
- hello() — método de prueba

```
import Funcionalidad.MDBExtendido;
import javax.jws.WebService;
import javax.jws.WebMethod;
import javax.jws.WebParam;

/**
 *
 * @author SirKy
 */
@WebService(serviceName = "WSBDatos")
public class WSBDatos {

    private MDBExtendido admin = new MDBExtendido();

    @WebMethod(operationName = "hello")
    public String hello(@WebParam(name = "name") String txt) {
        return "Hello " + txt + "!";
    }

    @WebMethod(operationName = "altas")
    public String altas(@WebParam(name = "sqlvalores") String sqlvalores) {
        return admin.registrarAlta("tbusuarios", sqlvalores);
    }

    @WebMethod(operationName = "baja")
    public String baja(@WebParam(name = "sqlcondicion") String sqlcondicion) {
        return admin.borrarRegistro("tbusuarios", sqlcondicion);
    }

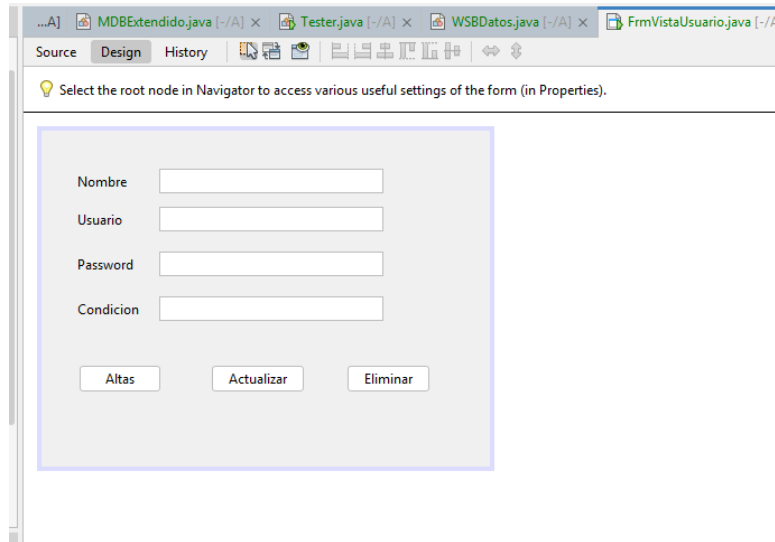
    @WebMethod(operationName = "modifica")
    public String modifica(
        @WebParam(name = "datos") String datos,
        @WebParam(name = "condicion") String condicion) {
        return admin.modificarRegistro("tbusuarios", datos, condicion);
    }
}
```

Este proyecto se desplegó en Apache Tomcat y se verificó accediendo al WSDL en <http://localhost:8080/WSBD/WSBDatos?WSDL>

Parte 2 — Frontend (Proyecto ClienteBd)

Creamos la interfaz gráfica usando la arquitectura **MVC (Modelo-Vista-Controlador)** con 3 paquetes:

Vista — FrmVistaUsuario.java Es el formulario gráfico con campos de texto y botones. No contiene lógica, solo muestra información al usuario.



Modelo — ModeloUsuario.java Se conecta al servicio web SOAP y expone sus métodos a la aplicación. Actúa como puente entre el controlador y el backend.

```
package Modelo;

/**
 *
 * @author SirKy
 */
import wsoap.WSBDatos;
import wsoap.WSBDatos_Service;

public class ModeloUsuario {

    WSBDatos_Service obj = new WSBDatos_Service();
    WSBDatos puerto = obj.getWSBDatosPort();

    public String registrarAlta(String sqlvalores) {
        return puerto.altas(sqlvalores);
    }

    public String eliminar(String condicion) {
        return puerto.baja(condicion);
    }

    public String modificar(String datos, String condicion) {
        return puerto.modifica(datos, condicion);
    }
}
```

Controlador — ControladorUsuario.java Es el núcleo de la arquitectura MVC. Implementa ActionListener para capturar los eventos de los botones, obtiene los datos de la Vista y los envía al Modelo. Nunca la Vista habla directamente con el Modelo.

```

- - -
package Controlador;

/**
 *
 * @author SirKy
 */

import Modelo.ModeloUsuario;
import Vista.FrmVistaUsuario;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.JOptionPane;

public class ControladorUsuario implements ActionListener {

    private FrmVistaUsuario _view;
    private ModeloUsuario _model;

    public ControladorUsuario(FrmVistaUsuario view, ModeloUsuario modelo) {
        this._model = modelo;
        this._view = view;
        // Registrar listeners en los botones de la Vista
        this._view.cmdAlta.addActionListener(this);
        this._view.cmdEliminar.addActionListener(this);
        this._view.cmdActualizar.addActionListener(this);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        String leyenda = "";

        if (e.getSource() == _view.cmdAlta) {
            // Alta: null para el autoincremental
            String valores = "null, "
                + _view.txtNombre.getText() + ", "
                + _view.txtUsuario.getText() + ", "
                + _view.txtpsw.getText() + ", 0";
        }
    }
}
```



```

        // Alta: null para el autoincremental
        String valores = "null, "
            + _view.txtNombre.getText() + ", "
            + _view.txtUsuario.getText() + ", "
            + _view.txtpsw.getText() + ", 0";
        leyenda = _model.registrarAlta(valores);
        JOptionPane.showMessageDialog(null, leyenda);
        limpiar();
    }

    if (e.getSource() == _view.cmdEliminar) {
        // Baja por condición escrita en txtCondicion, ej: idtbUsuarios=1
        String condicion = _view.txtCondicion.getText();
        leyenda = _model.eliminar(condicion);
        JOptionPane.showMessageDialog(null, leyenda);
        limpiar();
    }

    if (e.getSource() == _view.cmdActualizar) {
        // Modificar: en txtNombre va "campo='valor'", en txtCondicion la condición
        String datos = _view.txtNombre.getText();
        String condicion = _view.txtCondicion.getText();
        leyenda = _model.modificar(datos, condicion);
        JOptionPane.showMessageDialog(null, leyenda);
        limpiar();
    }
}

public void iniciar() {
    _view.setTitle("MVC BD - SOAP");
    _view.arranca();
}

public void limpiar() {
    _view.txtNombre.setText("");
    _view.txtUsuario.setText("");
    _view.txtpsw.setText("");
    _view.txtCondicion.setText("");
}
}

```

ClienteBd.java — clase principal con el main() que crea los tres objetos y lanza la aplicación.

```

import Controlador.ControladorUsuario;
import Modelo.ModeloUsuario;
import Vista.FrmVistaUsuario;

public class ClienteBd {
    public static void main(String[] args) {
        FrmVistaUsuario vista = new FrmVistaUsuario();
        ModeloUsuario modelo = new ModeloUsuario();
        ControladorUsuario ctrl = new ControladorUsuario(vista, modelo);
        ctrl.iniciar();
    }
}

```

PRUEBAS

Altas

MVC BD - SOAP

Nombre

Usuari

Passw

Condic

Mensaje

 Datos almacenados exitosamente.

```
MariaDB [bddatos]> SELECT * FROM tbusuarios;
```

idtbUsuarios	nombre	apPaterno	apMaterno	codigoAceso
2	Waffles	Cookies	porfis123	0
3	Mnuel	Manuelangas	153344	0
4	Samuel	Porky21	98473	0
5	AlvaroLEcVec	Alvarinho	Pass1234	0

```
4 rows in set (0.001 sec)
```

Actualizar

MVC BD - SOAP

Nombre

Usuari

Passw

Condic

Mensaje

 Registro modificado exitosamente.

```
MariaDB [bddatos]> SELECT * FROM tbusuarios;
```

idtbUsuarios	nombre	apPaterno	apMaterno	codigoAceso
2	Waffles	Cookies	porfis123	0
3	Mnuel	Manuelangas	153344	0
4	Samuel	Porky21	98473	0
5	Alvaro	Alvarinho	Pass1234	0

```
4 rows in set (0.001 sec)
```

```
MariaDB [bddatos]> |
```

Eliminar

MVC BD - SOAP

Nombre

Usuario

Passw

Condic

Mensaje

Registro eliminado exitosamente.

idtbUsuarios	nombre	apPaterno	apMaterno	codigoAceso
2	Waffles	Cookies	porfis123	0
3	Mnuel	Manuelangas	153344	0
4	Samuel	Porky21	98473	0

3 rows in set (0.001 sec)

MariaDB [bddatos]> |